

Lab Manual: Building a Complete Interactive Dashboard with Streamlit

Objective

By the end of this lab, you will:

- Upload and preview CSV data in a Streamlit app.
- Process the data with **Pandas**.
- Display **key metrics** interactively (sales, profit).
- Visualize results with **bar, line, and pie charts**.
- Export processed results as a CSV file.

Prerequisites

- Python 3.7+ installed.
- Install required libraries: `pip install streamlit pandas matplotlib`

Step 1: Create a New App File

Create a file named `app.py` and start with:

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

# Title and description
st.title("Interactive Sales Dashboard")
st.write("Upload a CSV file with sales and profit data to analyze and
```

```
visualize results.")
```

Explanation:

- `streamlit` → UI framework.
- `pandas` → data analysis.
- `matplotlib` → charting.
- `st.title / st.write` → explain what the app does.

Step 2: Upload CSV Data

```
# File uploader (only CSV allowed)
uploaded_file = st.file_uploader("Upload your sales CSV",
type=["csv"])

if uploaded_file is not None:
    # Read CSV into Pandas DataFrame
    df = pd.read_csv(uploaded_file)

    # Show a preview of the dataset
    st.write("Data Preview:")
    st.dataframe(df.head())
```

Explanation:

- `st.file_uploader` → lets users upload a CSV.
- `pd.read_csv` → loads data into a DataFrame.
- `st.dataframe` → shows first 5 rows interactively.

Step 3: Choose Metric for Analysis

```
# Dropdown for metric selection
metric = st.selectbox("Choose a metric:", ["sales", "profit"])
```

```

# Calculate total
total_value = df[metric].sum()
st.write(f"**Total {metric.capitalize()}:** {total_value}")

# Aggregate by region
agg_by_region = df.groupby("region")[metric].sum()
st.write(f"**{metric.capitalize()} by Region:**")
st.write(agg_by_region)

```

Explanation:

- `st.selectbox` → lets users pick between *sales* and *profit*.
- `df[metric].sum()` → calculates total.
- `groupby("region")[metric].sum()` → aggregates by region.
- `st.write` → displays results directly in the app.

Step 4: Choose Chart Type

```

# Dropdown for chart type
chart_type = st.selectbox("Choose a chart type:", ["Bar", "Line",
"Pie"])

fig, ax = plt.subplots()

if chart_type == "Bar":
    agg_by_region.plot(kind="bar", ax=ax, color="skyblue",
edgecolor="black")
    ax.set_ylabel(metric.capitalize())
    ax.set_title(f"{metric.capitalize()} by Region")

elif chart_type == "Line":
    agg_by_region.plot(kind="line", marker="o", ax=ax,
color="green")
    ax.set_ylabel(metric.capitalize())
    ax.set_title(f"{metric.capitalize()} by Region (Line Chart)")

```

```

elif chart_type == "Pie":
    agg_by_region.plot(kind="pie", autopct="%1.1f%%", ax=ax)
    ax.set_ylabel("") # Hide Y-axis for pie chart
    ax.set_title(f"{metric.capitalize()} Distribution by Region")

# Show chart in Streamlit
st.pyplot(fig)

```

Explanation:

- Users can switch between **Bar, Line, and Pie charts**.
- Chart updates instantly based on selection.
- `st.pyplot(fig)` renders the Matplotlib figure in Streamlit.

Step 5: Add Download Button

```

# Prepare processed results
processed_df = agg_by_region.reset_index()
processed_df.columns = ["Region", metric.capitalize()]

# Download button
csv = processed_df.to_csv(index=False).encode("utf-8")
st.download_button(
    label="Download Processed Data as CSV",
    data=csv,
    file_name=f"{metric}_by_region.csv",
    mime="text/csv",
)

```

Explanation:

- Converts aggregated data into a clean DataFrame.
- `st.download_button` → allows users to save results as a CSV.

Step 6: Run the App

Run the app in your terminal:

```
streamlit run app.py
```

Your browser will open at <http://localhost:8501>, showing:

1. File uploader
2. Data preview
3. Total metric value
4. Aggregated values by region
5. Interactive chart (bar, line, pie)
6. Download button for results

Verification Checklist

- CSV uploads successfully.
- Preview shows first rows.
- Dropdown switches between **sales** and **profit**.
- Total and per-region values update correctly.
- Chart type (bar, line, pie) changes dynamically.
- Download button exports processed CSV.

Summary

In this lab, you built a **complete interactive sales dashboard** in Streamlit that:

- Uploads and previews data.
- Processes metrics like **sales** or **profit**.
- Aggregates results by region.
- Visualizes insights with **bar, line, and pie charts**.
- Lets users **download results** for offline use.

This is now a **real-world style dashboard** that combines inputs, processing, and outputs, ready for business insights or further expansion.